Evntiqa Web & CMS Platform

Production Readiness & Technical Audit Report

Architecture, Security, Database Scalability & CMS Module Review

Date: November 2026	**Target Environments:** Staging & Production
Prepared For: Engineering Management	**Scope:** Editorial CMS, Authors, API, Database, Proxy

Executive Summary

This audit report summarizes critical vulnerabilities, architecture bugs, and scalability bottlenecks identified in the Evntiqa web platform codebase. Following the resolution of client-side loopback URLs that prevented blogs from appearing on staging and production, a full code audit was conducted on the **Editorial CMS**, **Authors & Bylines**, **Media Pipelines**, and **Deployment Infrastructure**. Key findings include a high risk of permanent image data loss on redeployment, upload size blocks in the reverse proxy, database connection pool exhaustion under modest traffic, and cascading author deletion issues.

1. Critical Production Breakage Risks

A. Uploaded Media is Destroyed on Every Container Redeployment (Data Loss)

Location: `docker-compose.yml (services.api)`
Impact: Critical (Permanent Data Loss)
Finding: In `docker-compose.yml`, while the Postgres database utilizes a persistent volume (`pgdata`), the backend API container mounts no volume for `/app/uploads`. All uploaded author avatar photos, blog cover pictures, and inline article assets are saved directly into the ephemeral container layer. The moment `docker compose up -d --build` runs or the server reboots, all files on disk vanish permanently. Database rows continue referencing these paths, resulting in permanent 404 broken images across the site.
Fix: Add an `uploads_data:/app/uploads` named volume to the `api` service.

B. Caddy Proxy Rejects Media Uploads with 413 Payload Too Large

Location: `deploy/Caddyfile (request_body max_size)`
Impact: High (Admin Operations Blocked in Production)
Finding: The Caddyfile enforces a strict global request body cap of **64 KB**: `request_body { max_size 64KB }`. Although the backend application configured an admin upload limit of 25 MB, Caddy intercepts incoming traffic ahead of FastAPI. As a result, uploading a standard cover image (e.g. 200 KB) or saving a rich blog post with embedded sections fails immediately at the reverse proxy with HTTP 413.
Fix: Update Caddy's `max_size` directive to 25 MB or remove it to delegate size validation to FastAPI's middleware.

C. SSRF Validator Blocks Editing Articles Containing Absolute Upload Paths

Location: `backend/app/services/media_service.py (to_local_upload_path)`
Impact: Medium-High (Publishing/Updating Blocked)
Finding: The function `to_local_upload_path()` only detects relative strings starting strictly with `/uploads/`. If a cover image or avatar URL contains the full hostname (e.g. `https://www.evntiqa.com/uploads/...` or `http://localhost:8000/uploads/...`), the parser treats it as an external image and triggers SSRF validation. The SSRF engine blocks private/loopback resolution, preventing the admin from saving the article.
Fix: Normalize URLs using `urlparse(url).path` before checking the `/uploads/` prefix.

2. Editorial Content & Author Module Deficiencies

A. Author Deletion Silently Strips Attribution on Published Blogs

Finding: In `backend/app/api/v1/admin/authors.py`, the deletion guard only checks if an author is credited on `ContentType.NEWS`. Because Evntiqa publishes `BLOG` articles, if an author who wrote multiple published blogs is deleted, the database foreign key (`ondelete='SET NULL'`) silently strips the author record. The published blogs become authorless with no warning.
Fix: Expand the deletion check to include all content types or block deletion if any published items are attached.

B. Strict 85% Fuzzy Match Algorithm Hard-Blocks Distinct Authors

Finding: In `backend/app/services/author_service.py`, name similarity is scored via `SequenceMatcher`. If a new author scores ≥ 85% similarity against any existing author, the API returns HTTP 409 Conflict unconditionally. Legitimate contributors with similar names (e.g. 'David Miller' vs 'Dave Miller') are completely blocked with no manual override flag available to admins.
Fix: Demote fuzzy match to an advisory warning in the UI with a confirmation override flag.

C. Missing Category Lifecycle (Categories Cannot Be Edited or Deleted)

Finding: The category management API (`admin/categories.py`) only implements GET and POST. There are no PUT or DELETE endpoints. Typographical errors in category names or slugs cannot be fixed through the portal, and obsolete categories cannot be retired without direct database intervention.
Fix: Add PUT (update) and DELETE (safe deprecation) endpoints for categories.

D. Topbar Refresh Button Inoperative on Authors Tab

Finding: In `admin.js`, clicking the topbar 'Refresh' button while on the Authors tab unconditionally calls `refreshEditorialList()` instead of `refreshAuthorsList()`. The authors table fails to reload.

3. Scalability & Database Performance Bottlenecks

Component / Area	Current State	Risk Under Production Load	Recommended Resolution

DB Connection Pool session.py	pool_size = 5 max_overflow = 5 (Total: 10 connections)	A burst of > 10 concurrent requests exhausts the pool, throwing QueuePoolTimeout 500 errors.	Increase pool_size to 20 and max_overflow to 20 for web scale.
Read View Counter content.py /by-slug	Every GET request executes UPDATE + db.commit() directly.	Search crawlers and viral spikes cause row-level write lock contention, exhausting connections.	Buffer increments in Redis or execute asynchronous background batch writes.
Article Search Query content.py list	Runs ContentItem.content.ilike('%q%') across full HTML body.	Forces PostgreSQL sequential table scan and toast decompression on 5 MB fields; spikes CPU to 100%.	Restrict search to title/brief, or introduce Postgres tsvector full-text index.
Author Duplicate Engine author_service.py	Loads all authors into Python memory on every author save.	O(N) in-memory fuzzy string comparisons create high latency as author count grows.	Rely on database unique constraints for email and exact normalized name.
Client-side Drafts admin.js auto-save	Saves entire Quill HTML string into browser localStorage every 1.2s.	Pasted screenshots (> 5 MB) cause QuotaExceededError and crash auto-save.	Catch QuotaExceededError and warn user when embedding raw base64 images.

4. Recommended Remediation Roadmap

Phase 1: Immediate Production Safeguards (Day 1)

• Mount `uploads_data:/app/uploads` in `docker-compose.yml` to protect images from deletion.

• Adjust `deploy/Caddyfile` request body size limit from 64KB to 25MB.

• Fix URL normalization in `media_service.py` to prevent self-hosted SSRF false alarms.

Phase 2: Admin Workflow & CMS Hardening (Sprint 1)

• Implement Category UPDATE and DELETE endpoints in the backend and admin dashboard.

• Extend author deletion check to cover BLOG content and prevent orphaned articles.

• Fix the 'Refresh' button on the Authors tab in `admin.js`.

• Add a confirmation override to the 85% author similarity rule.

Phase 3: High-Traffic Scaling & Database Optimization (Sprint 2)

• Scale SQLAlchemy connection pool to `pool_size=20, max_overflow=20`.

• Remove `content.ilike` from public search queries to eliminate full-table de-toasting scans.

• Decouple public view counter writes from the GET request lifecycle.

• Implement cache-busting version queries (`?v=...`) on static scripts.